



HOSPITAL DATABASE MANAGEMENT USING MYSQL



Project Overview

This project demonstrates the use of **MySQL DDL (Data Definition Language)** commands to create and modify a hospital patient database.

The project focuses on creating a Patients table and performing common database structure operations such as **CREATE, ALTER, MODIFY, RENAME, TRUNCATE, and DROP**.

It is designed to demonstrate practical knowledge of database table creation, schema modification, and data removal operations using MySQL.



Project Objectives

- Create a patient information table using SQL.
- Add new columns to an existing table.
- Modify the size of an existing column.
- Rename a database table.
- Understand the difference between TRUNCATE and DROP.
- Practice MySQL DDL commands.



Technologies Used

- **Database:** MySQL
- **Tool:** MySQL Workbench
- **Language:** SQL
- **Concept:** DDL – Data Definition Language

Questions & SQL Implementation

Question 1: CREATE Table

Scenario

Create a table named Patients with the following fields:

- PatientID
- PatientName
- Age
- Gender
- AdmissionDate

SQL Query

--- To create table

```
create table Patient(  
    Patient_ID varchar(20),  
    PatientName varchar(20),  
    Age int,  
    Gender enum('Female','Male'),  
    AdmissionDate date  
);
```

Expected Output

A new table named Patient is created successfully.

Question 2: ALTER – Add Column

Scenario

Add a new column Doctor Assigned with a data type of VARCHAR(50) to the Patients table.

SQL Query

```
--- alter table  
alter table patient  
add DoctorAssigned VARCHAR(50);
```

Expected Output

The Patients table now contains an additional column named DoctorAssigned.

Question 3: ALTER – Modify Column

Scenario

Increase the length of PatientName from VARCHAR(50) to VARCHAR(100).

SQL Query

```
--- alter table  
alter table patient  
modify column PatientName varchar(100);
```

Expected Output

The PatientName column now allows up to **100 characters**.

Question 4: RENAME Table

Scenario

Rename the table Patients to Patient_Info.

SQL Query

```
--- RENAME TABLE NAME  
Rename table patient TO Patient_info;
```

Expected Output

The table name is successfully changed from Patients to Patient_Info.

Question 5: TRUNCATE vs DROP

Scenario

At year-end:

1. Delete all patient records while keeping the table structure.
2. Later, remove the table permanently.

Step 1: TRUNCATE

SQL Query

--- truncate table

```
truncate TABLE patient_info;
```

Expected Output

- All records are removed.
- The table structure remains.
- The table can still be used for inserting new records.

Step 2: DROP

SQL Query

--- drop the table

```
DROP TABLE patient_info;
```

Expected Output

- The table is permanently removed.
- Both the table structure and its data are deleted.

TRUNCATE vs DROP

Feature	TRUNCATE	DROP
Removes Data	 Yes	 Yes
Removes Table Structure	 No	 Yes
Table Available After Command	 Yes	 No
Used For	Clearing all records	Permanently removing table
Can Insert Data Afterward	 Yes	 No

Example

TRUNCATE TABLE Patient_Info;

The table remains available.

DROP TABLE Patient_Info;

The table no longer exists.

Final Table Structure

Before applying TRUNCATE or DROP, the final table structure is:

Column Name	Data Type
PatientID	INT
PatientName	VARCHAR(100)
Age	INT
Gender	VARCHAR(10)
AdmissionDate	DATE
DoctorAssigned	VARCHAR(50)

Key Concepts Demonstrated

1. CREATE TABLE

Used to create a new table in the database.

2. ALTER TABLE

Used to change the structure of an existing table.

3. ADD COLUMN

Used to add a new column to an existing table.

4. MODIFY COLUMN

Used to change the data type or size of an existing column.

5. RENAME TABLE

Used to change the name of an existing table.

6. TRUNCATE

Used to remove all records while keeping the table structure.

7. DROP

Used to permanently delete the table and its structure.

Project Outcome

Through this project, I practiced essential **MySQL DDL operations** and gained hands-on experience in managing database table structures.

Output				
Action Output				
#	Time	Action	Message	
✓ 1	23:23:59	Create database Patient_DB	1 row(s) affected	
✓ 2	23:23:59	use Patient_DB	0 row(s) affected	
✓ 3	23:23:59	create table Patient(Patient_ID varchar(20), PatientName varchar(20), Age int, Gender enum('Fem...	0 row(s) affected	
✓ 4	23:23:59	alter table patient add DoctorAssigned VARCHAR(50)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 5	23:23:59	alter table patient modify column PatientName varchar(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 6	23:23:59	Rename table patient TO Patient_info	0 row(s) affected	
✓ 7	23:23:59	truncate TABLE patient_info	0 row(s) affected	
✓ 8	23:23:59	DROP TABLE patient_info	0 row(s) affected	

The project demonstrates how SQL can be used to:

- Create database tables

- Modify table structures
- Add new fields
- Change column definitions
- Rename tables
- Remove data while preserving structure
- Permanently delete database objects



Skills Demonstrated

- MySQL
- SQL
- DDL Commands
- CREATE TABLE
- ALTER TABLE
- ADD COLUMN
- MODIFY COLUMN
- RENAME TABLE
- TRUNCATE TABLE
- DROP TABLE
- Database Schema Management
- MySQL Workbench



Conclusion

This Hospital Database project provides practical experience with fundamental **MySQL DDL commands**. It demonstrates the ability to create, modify, rename, clear, and remove database tables while understanding the difference between TRUNCATE and DROP.

This project forms a strong foundation for learning more advanced SQL concepts such as **constraints, relationships, joins, DML commands, aggregate functions, subqueries, and database normalization**.